NXT WAVE™

WELCOME

NXT WAVE™

C++ STL: Map, Multi Map, Unordered Map

Agenda

C++ STL

C++ STL

Map

Map

A **map** is a data structure that **stores key-value pairs,** where each key is unique and maps to a specific value.

Use Case Example

*The **government** of India has to **maintain a list of** all the **citizens** of India.*

Let's see how we can store it in a Map.

Use Case Example

Data		
Name	**Gender**	**Place of Birth**
G Sashank	Male	Hyderabad
B Rani	Female	Mumbai
A Rahul	Male	Delhi
D Avinash	Male	Bangalore
⋮	⋮	⋮
G Sashank	Male	Chennai

Can we use **Name as key** and **other** parameters as **value** to **store** this data in a **map**?

Map

Use Case Example

Data		
Name	**Gender**	**Place of Birth**
G Sashank	Male	Hyderabad
B Rani	Female	Mumbai
A Rahul	Male	Delhi
D Avinash	Male	Bangalore
⋮	⋮	⋮
G Sashank	Male	Chennai

There might be a lot of people with **common names**.

That means name is not unique so it can't be used as Key.

Map

Use Case Example

Data			
Name	Gender	Place of Birth	Aadhaar Number
G Sashank	Male	Hyderabad	1254 8754 6915
B Rani	Female	Mumbai	2548 7415 6954
A Rahul	Male	Delhi	3644 1589 7458
D Avinash	Male	Bangalore	3258 7459 8216
⋮	⋮	⋮	⋮
G Sashank	Male	Chennai	3045 2424 8569

For each person, we can **store** their **Aadhaar number** as key and rest of the fields as value.

Aadhaar number **cannot be the same for two** people.

C++ STL

Map Container

*Now, If we want to search a single person, we can **search quickly with their Aadhar number.***

Example

Key	Value		
Aadhaar Number	Name	Gender	Place of Birth
1254 8754 6915	G Sashank	Male	Hyderabad
2548 7415 6954	B Rani	Female	Mumbai
3644 1589 7458	A Rahul	Male	Delhi
3258 7459 8216	D Avinash	Male	Bangalore
⋮	⋮	⋮	⋮
3045 2424 8569	G Sashank	Male	Chennai

Map container → Key - Value pair

This way, we can store data in a Map, **enabling faster lookups using the Aadhar number**.

C++ STL

Map Container

✓ Each **entry** in the **map** consists of a **unique key** and its **associated value**.

Map Container

Declarations

map< **datatype1** , **datatype2** > m;

Key

- Datatypes which supports “**<**” **comparator operator** can be used as **key**.
- All primitive data types(**except bool**) , **Strings**, **Pairs**.
- For **Custom Structures**/**Classes** to be used as **key** it must implement operator “**<**”.

Value

- Values can be of any datatype, including **primitive types**, **strings**, **vectors**, **sets**, **maps**, and **custom structs**.
- Supports complex structures, enabling **nested maps**, **nested pairs** etc.

Map Container

Some Example Declarations

✓ `map<int,int> m;`

✓ `map<pair<int,int>, map<int,int>> m;`

✓ `map<string,vector<int> > m;`

✓ `map<char,stack<int>> m;`

✗ `map<map<int,int>,string> m;`

✗ `map<bool,int> m;`

✗ `map<vector<int>, int> m;`

✗ `map<stack<int>,int> m`

Map

Declaration and Initialization

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
```

A map that has a **key** of **type int** and a **value** of **type string**.

Declaration and Initialization

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
```

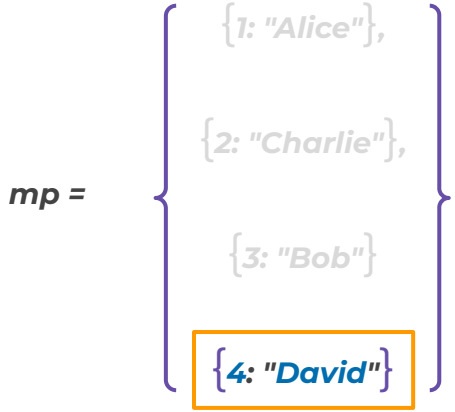
mp = { {1: "Alice"}, {2: "Charlie"}, {3: "Bob"} }

Initializing with key-value pairs.

Inserting Elements

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
mp.insert({4, "David"});
```

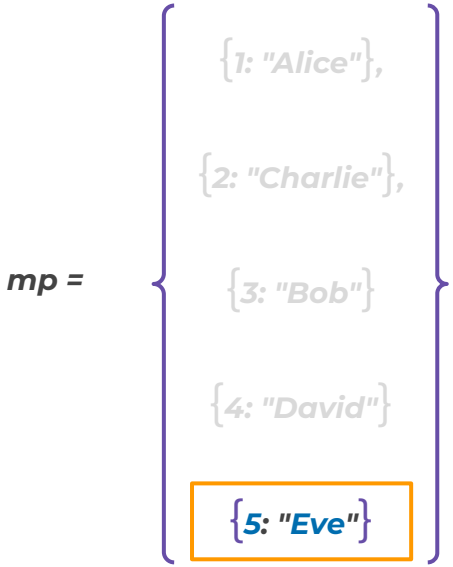


Inserting elements using insert( ).

Inserting Elements

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
mp.insert({4, "David"});
mp[5] = "Eve";
```

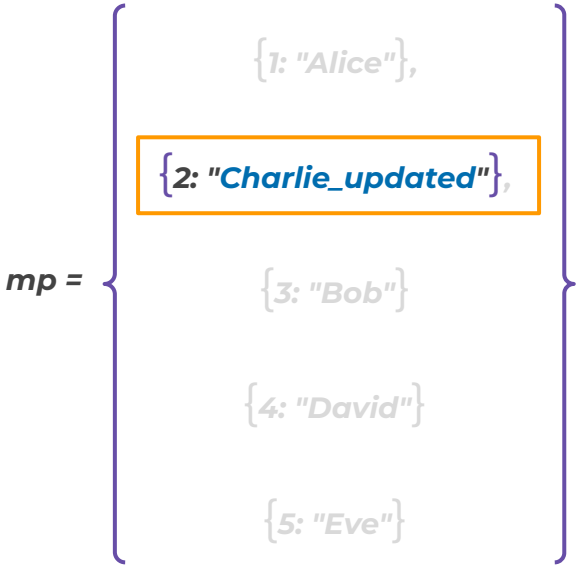


Inserting elements using operator[].

Inserting Elements

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
mp.insert({4, "David"});
mp[5] = "Eve";
mp[2] = "Charlie_updated";
```



If the key exists, the operator [] updates the value associated with that key.

Map

Checking if a Key Exists

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
if (mp.find(3) != mp.end()) {
    cout << "Key 3 exists";
} else {
    cout << "Key 3 not found";
}
```

mp.find(3) searches for **key 3** in the **map** and **returns an iterator** to it.

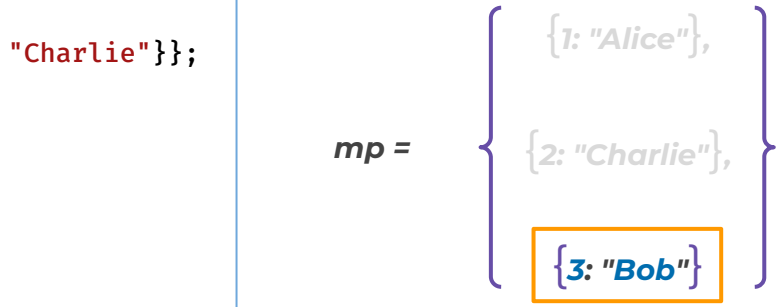
- ➔ If the **key exists**, the **iterator points** to the **key-value** pair.
- ➔ If the **key** is **not found**, it returns **mp.end()**.

Map

Checking if a Key Exists

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
if (mp.find(3) != mp.end()) {
    cout << "Key 3 exists";
} else {
    cout << "Key 3 not found";
}
```



Map

Checking if a Key Exists

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
if (mp.find(3) != mp.end()) {
   cout << "Key 3 exists";
} else {
   cout << "Key 3 not found";
}
```

Output

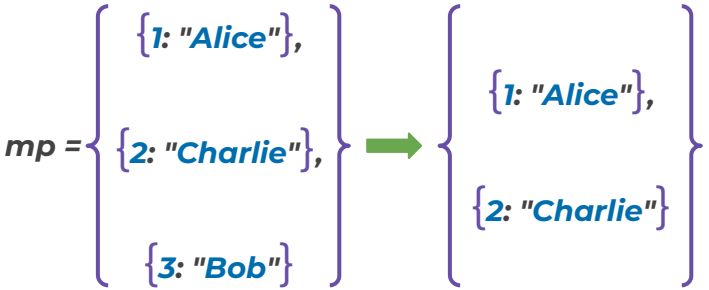
Key 3 exists

Map

Removing Elements

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
mp.erase(3);
```



Getting the Size of the Map

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
cout << mp.size();
```

mp = { *1: "Alice"*, *2: "Charlie"*, *3: "Bob"* }

Outputs the number of key-value pairs in the map.

Getting the Size of the Map

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
cout << mp.size();
```

Output

```
3
```

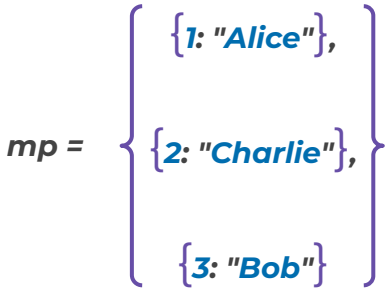
Outputs the number of key-value pairs in the map.

Iterating Over a Map

Using Range-Based For Loop

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto element : mp) {
   cout << element.first << " : " << element.second << endl;
}
```



Iterating Over a Map

Using Range-Based For Loop


```cpp
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto element : mp) {
    cout << element.first << " : " << element.second << endl;
}
```

element.first refers to the **key** of **each element** in the **map** during iteration.

Iterating Over a Map

Using Range-Based For Loop

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto element : mp) {
   cout << element.first << " : " << element.second << endl;
}
```

element.second refers to the value of each element in the map during iteration.

Iterating Over a Map

Using Range-Based For Loop

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto element : mp) {
    cout << element.first << " : " << element.second << endl;
}
```

Output

```
1 : Alice
2 : Charlie
3 : Bob
```

Clearing the Map

C++

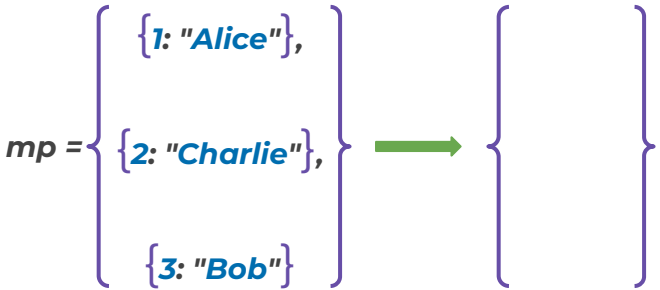
```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

mp.clear();
```

mp = {1: "Alice"}, {2: "Charlie"}, {3: "Bob"}

Clearing the Map

C++

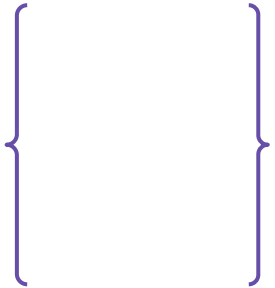
```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
mp.clear();
```



Checking If the Map is Empty

C++

```
map<int, string> mp;
if (mp.empty()) {
    cout << "Map is empty";
}
```

mp = 

Checking If the Map is Empty

C++

```
map<int, string> mp;

if (mp.empty()) {
    cout << "Map is empty";
}
```

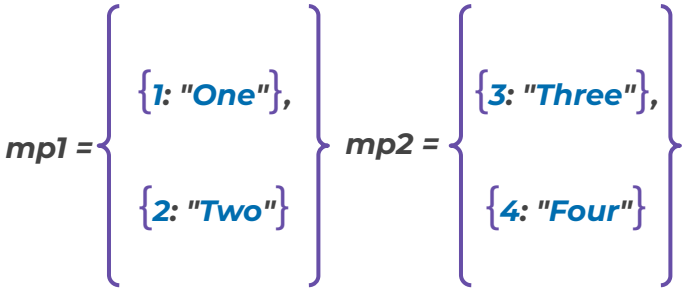
Output

Map is empty

Swapping Two Maps

C++

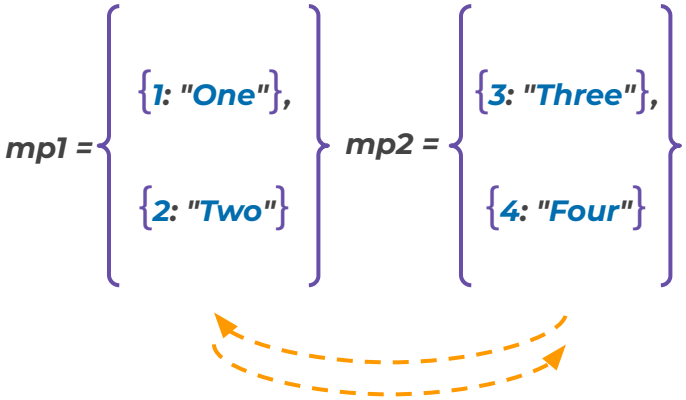
```
map<int, string> mp1 = {{1, "One"}, {2, "Two"}};
map<int, string> mp2 = {{3, "Three"}, {4, "Four"}};
```



Swapping Two Maps

C++

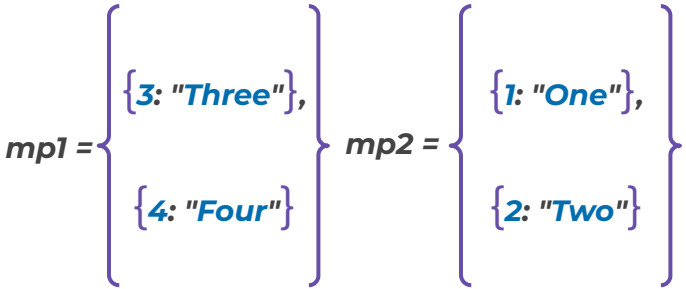
```
map<int, string> mp1 = {{1, "One"}, {2, "Two"}};
map<int, string> mp2 = {{3, "Three"}, {4, "Four"}};
mp1.swap(mp2);
```



Swapping Two Maps

C++

```
map<int, string> mp1 = {{1, "One"}, {2, "Two"}};
map<int, string> mp2 = {{3, "Three"}, {4, "Four"}};
mp1.swap(mp2);
```



Quiz Time!

Multimap

Multimap

Multi Map

A **multimap** is a modified version of **map** that allows **duplicate keys** while **maintaining sorted** order.

Multimap

Declaration and Initialization

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};
```

$$mm = \left\{ \begin{matrix} \{1: \text{"Alice"}\}, \{2: \text{"Bob"}\}, \\ \{2: \text{"Charlie"}\}, \{3: \text{"David"}\} \end{matrix} \right\}$$

Duplicate key, allowed.

Multimap

Inserting Elements

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};
mm.insert({3, "Eve"});
```

mm = {1: "Alice"}, {2: "Bob"}, {2: "Charlie"}, {3: "David"}, **{3: "Eve"}**

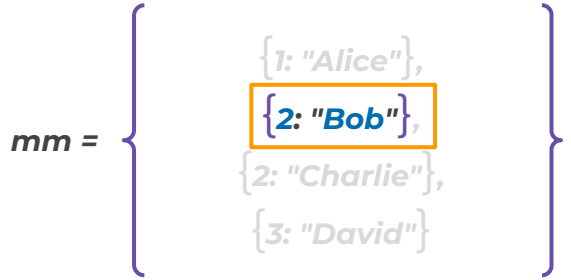
Insert using insert( ), duplicate keys allowed.

NXT WAVE™

Checking if a Key Exists

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

if (mm.find(2) != mm.end()) {
    cout << "Key 2 exists";
} else {
    cout << "Key 2 not found";
}
```



In a multimap, **mp.find(x)** searches for key x.

- If the key exists, returns the iterator pointing to the first matching key-value pair.
- If the key is not found, returns **mp.end()**.

Multimap

Checking if a Key Exists

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

if (mm.find(2) ! = mm.end()) {
   cout << "Key 2 exists";
} else {
   cout << "Key 2 not found";
}
```

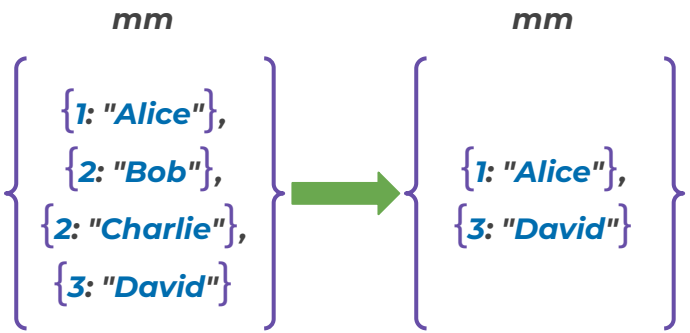
Output

Key 2 exists

Removing all occurrence of a key

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

mm.erase(2);
```

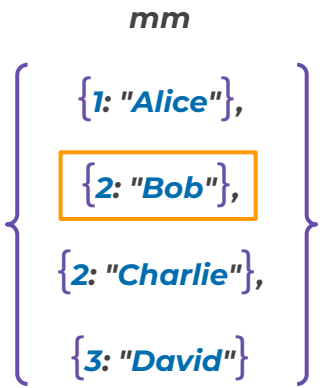


mm.erase(x) in multimap - Removes all key-value pairs with key x.

Removing first occurrence of a key

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

auto it = mm.find(2);
```

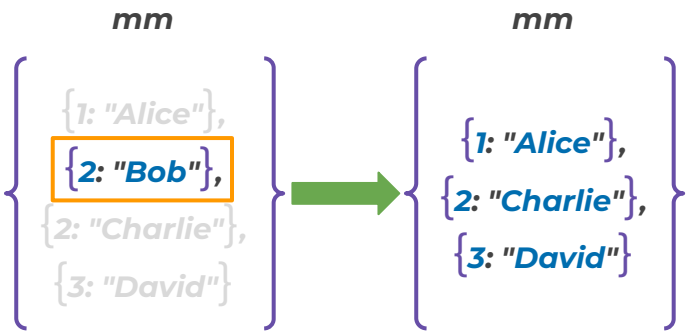


Finds the first occurrence of key 2.

Removing first occurrence of a key

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

auto it = mm.find(2);
if (it != mm.end()) {
   mm.erase(it);
}
```



Finds the first occurrence of key 2.

Multimap

Counting Elements

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

mm.count(2);
```

Output

2

The **count function in a multimap** can be used to count the number of elements with a specific key.

Size of the Multimap

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

cout << mm.size();
```

Outputs the number of key-value pairs in the multimap.

mm

{1: "Alice"},
{2: "Bob"},
{2: "Charlie"},
{3: "David"}

Size of the Multimap

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

cout << mm.size();
```

Output

4

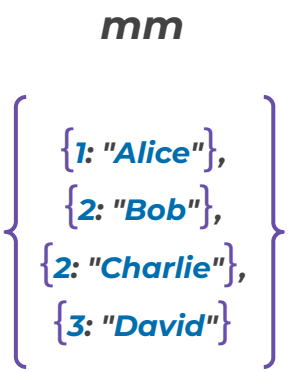
Outputs the number of key-value pairs in the multimap.

Multimap

Using Range-Based For Loop

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

for (auto element : mm) {
    cout << element.first << ": " << element.second << endl;
}
```



Multimap

Using Range-Based For Loop

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

for (auto element : mm) {
    cout << element.first << ": " << element.second << endl;
}
```

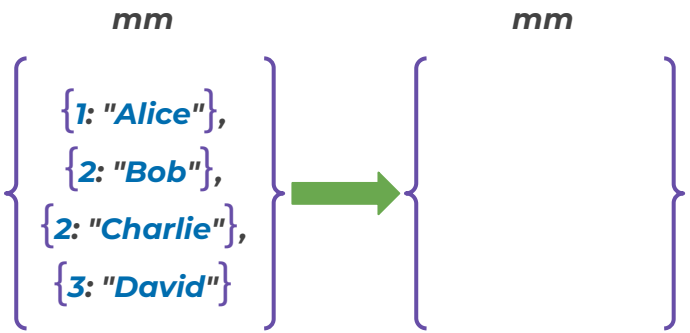
Output

```
1: Alice
2: Bob
2: Charlie
3: David
```

Clearing the Multimap

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

mm.clear();
```

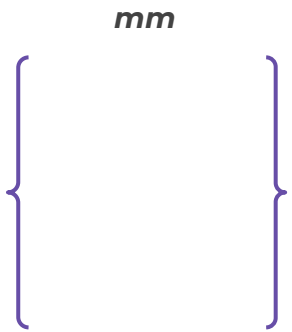


Removes all key-value pairs.

Checking If the Multimap is Empty

C++

```
multimap<int, string> mm;

if (mm.empty()) {
   cout << "Multimap is empty";
}
```



mm.empty() returns **1** if the multimap is **empty**; otherwise, **it returns 0**.

Checking If the Multimap is Empty

C++

```
multimap<int, string> mm;

if (mm.empty()) {
    cout << "Multimap is empty";
}
```

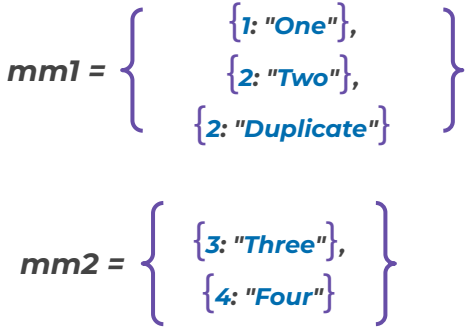
Output

Multimap is empty

Swapping Two Multimaps

C++

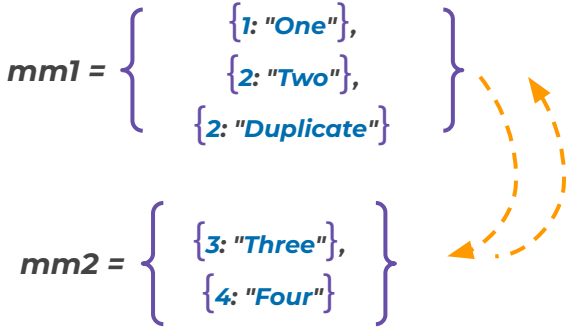
```
multimap<int, string> mm1 = {{1, "One"}, {2, "Two"}, {2, "Duplicate"}};
multimap<int, string> mm2 = {{3, "Three"}, {4, "Four"}};
```



Swapping Two Multimaps

C++

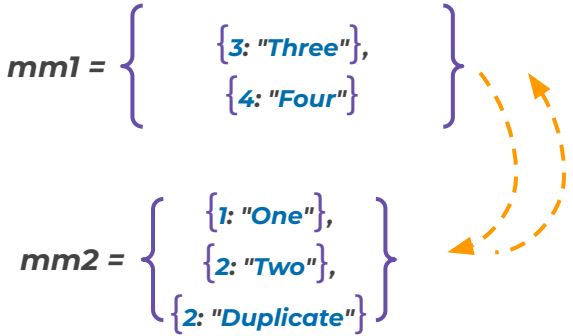
```
multimap<int, string> mm1 = {{1, "One"}, {2, "Two"}, {2, "Duplicate"}};
multimap<int, string> mm2 = {{3, "Three"}, {4, "Four"}};
mm1.swap(mm2);
```



Swapping Two Multimaps

C++

```
multimap<int, string> mm1 = {{1, "One"}, {2, "Two"}, {2, "Duplicate"}};
multimap<int, string> mm2 = {{3, "Three"}, {4, "Four"}};
mm1.swap(mm2);
```



Quiz Time!

Unordered Map

Unordered Map

unordered_map

unordered_map

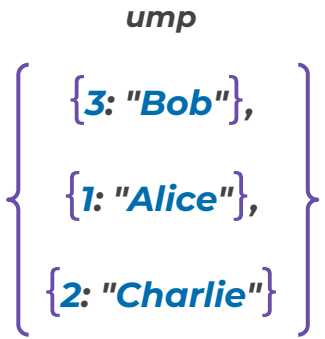
An unordered_map is a data structure that **stores key-value pairs** in no **particular order with unique keys** .

Unordered Map

Declaration and Initialization

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
```



Initializing with key-value pairs.

Unordered Map

Inserting Elements

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
ump.insert({4, "David"});
```

ump

{3: "Bob"},

{1: "Alice"},

{2: "Charlie"},

{4: "David"}

Inserting elements using insert().

Unordered Map

Inserting Elements

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
ump.insert({4, "David"});
ump[5] = "Eve";
```

ump

{3: "Bob"},
{1: "Alice"},
{2: "Charlie"},
{4: "David"},
{5: "Eve"}

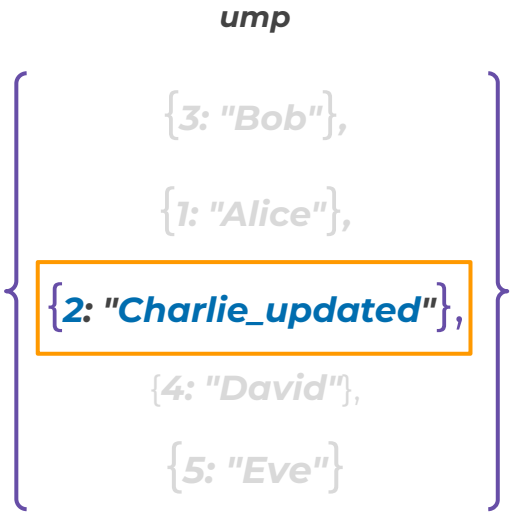
Inserting elements using operator[ ].

Unordered Map

Inserting Elements

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};
ump.insert({4, "David"});
ump[5] = "Eve";
ump[2] = "Charlie_updated";
```



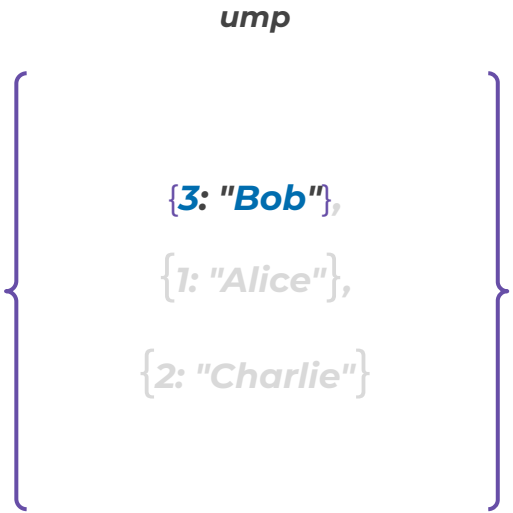
Updates existing key-value pair.

Unordered Map

Checking if a Key Exists

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

if (ump.find(3) != ump.end()) {
    cout << "Key 3 exists";
} else {
    cout << "Key 3 not found";
}
```



In an unordered_map, **mp.find(x)** searches for key **x**.

- If the key exists, returns the iterator pointing to the corresponding key-value pair.
- If the key is not found, returns **mp.end()**.

Unordered Map

Checking if a Key Exists

C++

```cpp
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

if (ump.find(3) != ump.end()) {
    cout << "Key 3 exists";
} else {
    cout << "Key 3 not found";
}
```

Output

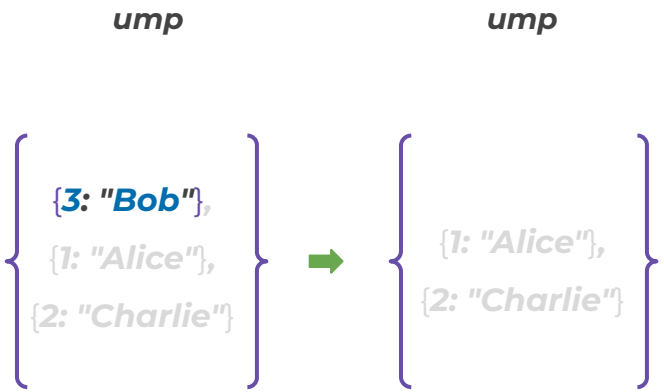
```
Key 3 exists
```

Unordered Map

Removing Elements

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

ump.erase(3);
```



Removes key 3.

Unordered Map

Size of the Unordered_map

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

cout << ump.size();
```

ump

{3: "Bob"},
{1: "Alice"},
{2: "Charlie"}

Outputs the number of key-value pairs in the unordered_map.

Unordered Map

Size of the Unordered_map

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

cout << ump.size();
```

Output

3

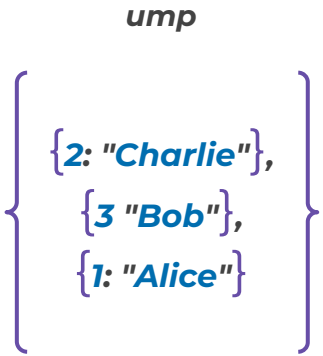
Outputs the number of key-value pairs in the unordered_map.

Unordered Map

Using Range-Based For Loop

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto element : ump) {
    cout << element.first << ": " << element.second << endl;
}
```



Unordered Map

Using Range-Based For Loop

C++

```cpp
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto element : ump) {
    cout << element.first << ": " << element.second << endl;
}
```

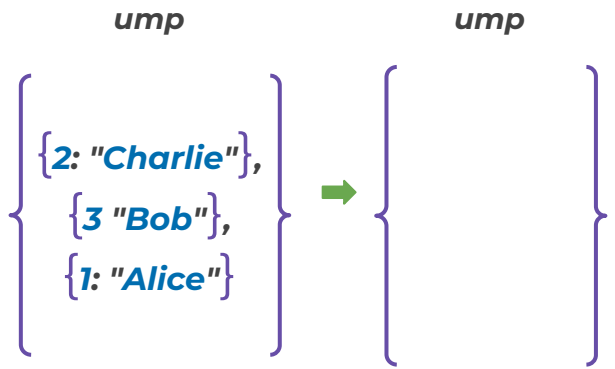
Output

2: Charlie
3: Bob
1: Alice

Clearing the Unordered_map

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

ump.clear();
```

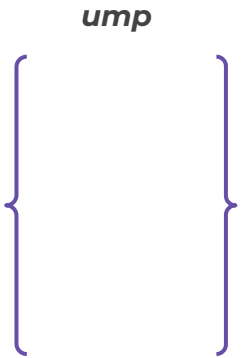


Removes all key-value pairs.

Checking If the Unordered Map is Empty

C++

```
unordered_map<int, string> ump;

if (ump.empty()) {
    cout << "Unordered map is empty";
}
```



ump.empty() returns **1** if the unordered_map is **empty**;

otherwise, **it returns 0**.

Checking If the Unordered Map is Empty

C++

```
unordered_map<int, string> ump;

if (ump.empty()) {
    cout << "Unordered map is empty";
}
```

Output

Unordered map is empty

Swapping Two Unordered Maps

C++

```
unordered_map<int, string> ump1 = {{1, "One"}, {2, "Two"}};
unordered_map<int, string> ump2 = {{3, "Three"}, {4, "Four"}};
```

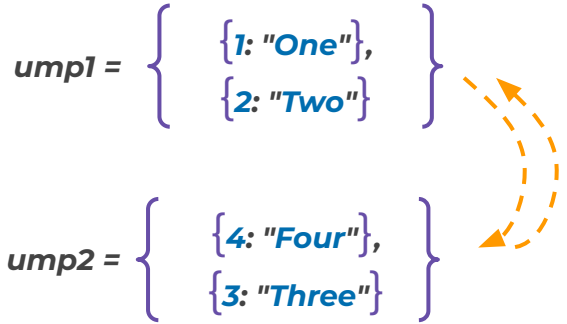
ump1 = { {*1: "One"*}, {*2: "Two"*} }

ump2 = { {*4: "Four"*}, {*3: "Three"*} }

Swapping Two Unordered Maps

C++

```
unordered_map<int, string> ump1 = {{1, "One"}, {2, "Two"}};
unordered_map<int, string> ump2 = {{3, "Three"}, {4, "Four"}};
ump1.swap(ump2);
```



Unordered Map

Swapping Two Unordered Maps

C++

```
unordered_map<int, string> ump1 = {{1, "One"}, {2, "Two"}};
unordered_map<int, string> ump2 = {{3, "Three"}, {4, "Four"}};
ump1.swap(ump2);
```

ump1 = { *{4: "Four"}*, *{3: "Three"}* }

ump2 = { *{1: "One"}*, *{2: "Two"}* }

Quiz Time!

Key Takeaways

- ***Map***

  stores **unique key-value** pairs in sorted order and allows fast lookups using keys.

- ***Multimap***

  allows **duplicate keys** and maintains sorted order for all **key-value** pairs.

Key Takeaways

- ***Unordered Map***

  stores **unique keys** with faster access but no specific order.

- ***Common operations***

  insert(), **find()**, **erase()**, **size()**, **clear()**, and looping using range or iterators.

NXT WAVE™

THANK YOU

NXT WAVE™

ALL THE BEST

Iterating Over a Map

Using Iterators

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto it = mp.begin(); it ! = mp.end(); it++) {
   cout << it→first << " : " << it→second << endl;
}
```

mp = { *1: "Alice"*, *2: "Charlie"*, *3: "Bob"* }

Iterating Over a Map

Using Iterators

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto it = mp.begin(); it ! = mp.end(); it++) {
   cout << it→first << " : " << it→second << endl;
}
```

it->first gives the key of the current map element during iteration.

Iterating Over a Map

Using Iterators

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto it = mp.begin(); it ! = mp.end(); it++) {
   cout << it→first << " : " << it→second << endl;
}
```

it->second gives the **value** of the **current** map **element** during **iteration**.

Iterating Over a Map

Using Iterators

C++

```
map<int, string> mp = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto it = mp.begin(); it ! = mp.end(); it++) {
    cout << it→first << " : " << it→second << endl;
}
```

Output

```
1 : Alice
2 : Charlie
3 : Bob
```

Iterating Over a Multimap

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

for (auto it = mm.begin(); it != mm.end(); it++) {
    cout << it->first << ": " << it->second << endl;
}
```

mm

{1: "Alice"},
{2: "Bob"},
{2: "Charlie"},
{3: "David"}

Iterating Over a Multimap

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};

for (auto it = mm.begin(); it != mm.end(); it++) {
    cout << it->first << ": " << it->second << endl;
}
```

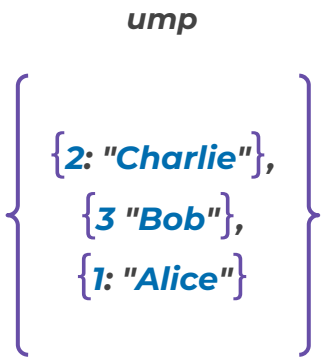
Output

```
1: Alice
2: Bob
2: Charlie
3: David
```

Iterating Over an Unordered Map

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto it = ump.begin(); it != ump.end(); it++) {
    cout << it->first << ": " << it->second << endl;
}
```



Iterating Over an Unordered Map

C++

```
unordered_map<int, string> ump = {{1, "Alice"}, {3, "Bob"}, {2, "Charlie"}};

for (auto it = ump.begin(); it != ump.end(); it++) {
    cout << it->first << ": " << it->second << endl;
}
```

Output

```
2: Charlie
3: Bob
1: Alice
```

Multimap

Inserting Elements

C++

```
multimap<int, string> mm = {{1, "Alice"}, {2, "Bob"}, {2, "Charlie"}, {3, "David"}};
mm.insert({3, "Eve"});
mm[2] = "Frank";
```

mm =

{1: "Alice"},
{2: "Bob"},
{2: "Charlie"},
{2: "Frank"},
{3: "David"},
{3: "Eve"}

Duplicate keys, allowed.

Word Frequency Counter

Problem Statement

- You are building a **word frequency** counter. Given a paragraph of **text**, you need to **count** how many times **each word appears**.

Word Frequency Counter

Example

Input

"the quick brown fox jumps over the lazy dog the quick fox"

Output

brown-1
dog-1
fox-2
jumps-1
lazy-1
over-1
quick-2
the-3

Word Frequency Counter

Example

["the", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog", "the", "quick", "fox"].

Count the frequency of each word:

***Brown* : 1** occurrences

***dog* : 1** occurrences

***fox* : 2** occurrence

***jumps* : 1** occurrences

***lazy* : 1** occurrence

***over* : 1** occurrence

***quick* : 2** occurrence

***the* : 3** occurrence

Word Frequency Counter

Code

C++

```
#include<bits/stdc++.h>
using namespace std;
class WordFrequencyCounter {
public:
   void countWords(map<string, int>& wordCount, string& paragraph) {
        stringstream ss(paragraph);
        string word;
        while (ss >> word) {
            wordCount[word]++;
        }
        for (auto& pair : wordCount) {
            cout << pair.first <<" - " << pair.second << endl;
        }
   }
};
```

Multimap Operations

Problem Statement

You are given a **multimap** that stores **key-value pairs**. Your task is to perform following operations on it:

- **Insertion**: Add a key-value pair to the multimap.
- **Find**: Check if a key exists in the multimap. Print "**Found**" if found, otherwise print "**Not Found**".
- **Count**: Get the number of values associated with a given key and return the count.
- **Erase**: Remove all occurrences of a key from the multimap.

Multimap Operations

Example

Input

Multimap:
[(apple, 18), (banana, 28), (banana, 30), (cherry, 48)]

Operations:
insert banana 50
find mango
count banana
erase banana

Output

Not Found
3

Multimap Operations

Example

- Insert **adds** (banana 🍌, 50), keeping the order intact.
- Find searches for "**mango** 🥭 " and returns "**Not Found**".
- Count returns 3 as "**banana**" appears **three times**. 🍌🍌🍌
- Erase **removes** all "**banana**" entries, **leaving** only "**apple**" and "**cherry**".

Code

C++

```
#include <bits/stdc++.h>
using namespace std;

class MultiMap {
public:
    multimap<string, int> mm;
    void insert(multimap<string, int> &mm, string& key, int value) {
        mm.insert({key, value});
    }
```

Multimap Operations

Code

C++

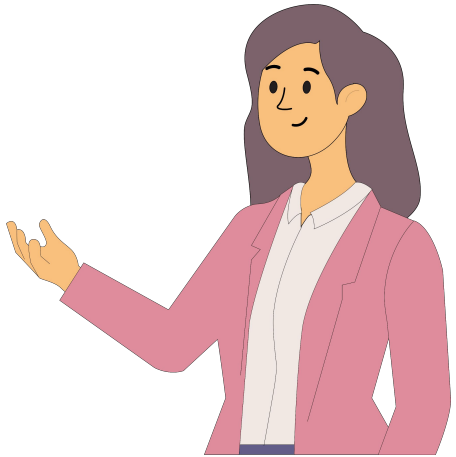
```
void find(multimap<string, int> &mm, string& key) {
   if (mm.find(key) != mm.end()) {
        cout << "Found" << endl;
   }
   else {
        cout << "Not Found" << endl;
   }
}
void count(multimap<string, int> &mm, string& key) {
   cout << mm.count(key);
}
void erase(multimap<string, int> &mm, string& key) {
   mm.erase(key);
}
};
```

Unordered Map

Problem Statement

You're given an unordered map where the key represents the number and the value represents its frequency. Your task is to perform the following operations:

- **Insertion**: Add/Update element into the unordered_map with a frequency.
- **Erase**: Remove an element from the unordered map.
- **Find**: Search key in the unordered map and print "**Found**" if present, "**Not Found**" otherwise.



Unordered Map

Problem Statement

Input

unordered_map: [1, 1, 2, 2, 2, 3, 4, 4, 4, 4]

Operations:
Insert 5 with frequency 3
Erase 2
Find 3

Output

Found

Unordered Map

Explanation

Insertion adds 5 with frequency 3.

Result

{1: 2, 2: 3, 3: 1, 4: 5, 5: 3}.

Erase removes all occurrences of 2.

Result

{1: 2, 3: 1, 4: 5, 5: 3}.

Find checks if 3 exists in the unordered_map.

Result

Found

Unordered Map

Code

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
class UnorderedMap {
public:
    void insertElement(unordered_map<int, int> &ump, int value, int frequency) {
        ump[value] += frequency;
    }
    void eraseElement(unordered_map<int, int> &ump, int value) {
        ump.erase(value);
    }
    bool findElement(unordered_map<int, int> &ump, int value) {
        return ump.find(value) != ump.end();
    }
};
```